LiDAR Perception Pipeline

Study guide: methods, numbers, algorithms and C++

E-Agle Formula Student Driverless — Perception Task (LiDAR path) — October 2026

Victor Gabriele De Matteis

This guide prepares the oral presentation of a LiDAR-only perception pipeline for a Formula Student Driverless car, written in C++20 with PCL, Eigen, NLopt and Qt. The pipeline loads a 24,000-point scan, removes the ground and walls by statistical filtering and region growing, clusters the remainder, recognises cones by ICP alignment with a synthetic cone model, splits the cones into two track edges, optimises a minimum-curvature racing line with BOBYQA, and estimates the vehicle displacement between two scans with ICP. For every stage the guide gives the method, the formulas, the algorithm in pseudocode, the C++ that implements it and the numbers obtained by re-running the code. Each statistic is justified by how it was measured. The limitations an examiner is likely to probe are stated plainly, with prepared answers.

Contents

1 How to use this guide **2**
1.1 The 60-second pitch . . . 2

2 Context: the task, the data and where it sits in computer vision **3**
2.1 The task . . . 3
2.2 The data . . . 3
2.3 Classical geometric pipeline versus learning . . . 3

3 Architecture and coordinate frames **4**
3.1 Pipeline overview . . . 4
3.2 Homogeneous transforms . . . 4
3.3 The levelling transform collapses to one rotation . . . 4

4 Level 1 — Loading and preprocessing **5**
4.1 Pass-through (height crop) . . . 5
4.2 Statistical outlier removal (SOR) . . . 5

5 Level 2 — Segmentation: removing walls and clustering **6**
5.1 Surface normals by PCA . . . 6
5.2 Region growing . . . 6
5.3 Euclidean cluster extraction . . . 6

6 Level 3 — Classifying cones and obstacles **7**
6.1 Cheap geometric gates first . . . 7
6.2 The ideal cone model . . . 7
6.3 ICP alignment as a shape test . . . 7
6.4 Results for the nine candidates . . . 8
6.5 Point density versus range . . . 9
6.6 Cone centres . . . 9
6.7 Obstacles . . . 9

7 Level 4a — Separating left and right edges **9**
7.1 The turning angle . . . 9

8 Level 4b — Racing line by constrained optimisation **10**
8.1 Formulation . . . 10
8.2 Where the curvature formula comes from . . . 11
8.3 BOBYQA . . . 11
8.4 Measured results on the scene . . . 12
8.5 Validation on a toy track: NLopt versus CasADi/IPOPT . . . 12

9 Level 5 — LiDAR odometry with ICP **13**
9.1 ICP . . . 13
9.2 Preprocessing (measured counts) . . . 13
9.3 Result . . . 13
9.4 From the transform to the vehicle displacement . . . 13
9.5 How good is this estimate? . . . 14

10 Bonus — Interactive GUI **15**

11 C++: difficult patterns, explained **16**
11.1 Shared pointers, aliasing and "const" that is not const . . . 16
11.2 In-place filtering is safe in PCL . . . 16
11.3 Keep-organised extraction: NaN as a tombstone . . . 16
11.4 Structured bindings and aggregate returns (C++17) . . . 16
11.5 Argsort with `iota` + `sort` + a capturing lambda . . . 17
11.6 `std::vector<char>` instead of `std::vector<bool>` . . . 17
11.7 Lambdas returning references need an explicit return type . . . 17
11.8 A C callback with `void*` user data (NLopt C API) . . . 17
11.9 A function template with a dependent type . . . 17
11.10 Eigen: `rotate` post-multiplies . . . 18
11.11 Default arguments and duplicate definitions across files . . . 18
11.12 Templates and precompiled instantiations: why Debug ≈ Release . . . 18
11.13 Qt: signals, slots, moc and the generated UI . . . 18
11.14 Small traps that are easy to miss . . . 18

12 Statistics: what each number means and how it was obtained **18**
12.1 Engineering footprint (from `STATS.md`) . . . 19
12.2 Runtime statistics (odometry) . . . 19
12.3 Runtime statistics (cone pipeline) . . . 20

13 Limitations and what to do next **20**

14 Question-and-answer drill **20**

15 Formula sheet **22**

1 How to use this guide

Sections 2–3 set the context and the coordinate frames. Sections 4–9 follow the five task levels in pipeline order. Section 10 covers the bonus GUI, Section 11 the C++ patterns, Section 12 the statistics, and Section 14 a question-and-answer drill. Three kinds of box recur:

Green — numbers worth memorising.

Orange — likely questions, with a prepared answer.

Red — places where the code does something different from what the README says, or where a result is weaker than it looks. Read these before presenting; it is far better to raise them yourself than to have them found.

Where the numbers come from. Counts and timings marked "measured" were obtained on 2026-10-07 by building the project sources headless (outside the repository) and calling the same functions with the GUI's default parameters, on the development machine (Mac15,12, Apple M3, 8 logical CPUs, macOS 15.6.1, PCL 1.15 from Homebrew, built without OpenMP). The odometry figures reproduce exactly the output stored in `screenshots/Graphical Material.pdf` (fitness 0.49974, North 0.2331, East -0.5432), which shows the code is deterministic on this data.

1.1 The 60-second pitch

> "I chose the LiDAR path and implemented all five levels plus the interactive GUI in C++20 with PCL. A 24,000-point scan is levelled with one rotation, cropped to a height band, and cleaned with statistical outlier removal. Planar structures — the walls — are removed by region growing on surface normals. Euclidean clustering then gives 67 candidate objects. Cheap geometric gates (size, ground contact, height) reduce these to 9. Each candidate is aligned to a synthetic cone with ICP and accepted if the residual and the required rotation are small: 6 cones. The

cones are split into two edges by a greedy nearest-neighbour rule with a turning-angle gate. A racing line is found by optimising lateral offsets from the centreline under track-width bounds with the derivative-free BOBYQA method. For odometry I level both scans, filter them and register them with ICP, which gives a planar displacement of about 0.59 m. The full cone pipeline runs in about 0.5 s, and the odometry ICP in about 2.3 s on an M3."

Numbers to remember

Input scans	24,000 points each (`cones.pcd`, `first.pcd`, `second.pcd`)
Cone path funnel	24,000 → 9,089 (height crop) → 8,251 (SOR) → 4,025 (walls removed)
Clusters → candidates → cones	67 → 9 → **6** cones (332 points)
Cone ICP residual (accepted)	fitness $4-11 \times 10^{-4}$ m^2 ⇒ RMS 2.0–3.3 cm
Yaw gate	13°; accepted 1.2–12.5°, rejected 17.5–45.3°
Left / right	3 + 3 cones
Racing line	$N = 3$ offsets, BOBYQA, 78 evaluations, `XTOL_REACHED`, 200 drawn points
Cone pipeline time	493 ± 28 ms end-to-end (≈ 2 Hz); region growing dominates
Odometry ICP	70 iterations (cap reached), fitness 0.49974 m^2, 7.56° rotation
Odometry displacement	0.543 m forward, 0.233 m lateral ⇒ 0.591 m planar
Odometry ICP time	2,291 ms (STATS.md); reproduced $2{,}278 \pm 74$ ms; 0.44 Hz
Resources	94.6–94.9 % CPU (≈ one core), 110–113 MiB peak RSS

2 Context: the task, the data and where it sits in computer vision

2.1 The task

The E-Agle perception task asks for five levels on either the image or the LiDAR path, plus a bonus GUI, within two weeks. On the LiDAR path, levels 1–4 use `cones.pcd` and level 5 uses `first.pcd` and `second.pcd`.

Level	Requirement	What was built (main file)
1	Load, display, preprocess	PCD loading, levelling transform, pass-through, SOR (`get_cones_cloud.cpp`)
2	Detect cones	Region growing to remove walls, Euclidean clustering (`get_cones_cloud.cpp`)
3	Classify cones vs. obstacles	Geometric gates + ICP against a cone model (`cones_recog.cpp`); obstacles as the remainder (`get_obstacle_cloud.cpp`)
4	Track edges / racing line	Greedy left/right split (`distinguish_left_right.cpp`); NLopt BOBYQA racing line (`racingline_nlopt.cpp`)
5	Odometry	Levelling + SOR + ICP between two scans (`OdometryWithProcessing/main.cpp`)
Bonus	Interactive visualisation	Qt + VTK viewer with sliders (`pclviewer.cpp`, `pclviewer.ui`)

Table 1: Task levels and where each is implemented.

2.2 The data

All three PCD files are binary, unorganised (`HEIGHT 1`) and contain 24,000 points with the fields `x y z intensity tag line`. The code loads them as `pcl::PointXYZ`, so intensity, tag and line are discarded. The extra `tag` and `line` fields are those written by Livox-type sensors, and 24,000 points per frame is consistent with a 240 kpts/s sensor at 10 Hz. This is an inference from the file format, not a documented fact, but it matters for one argument: at 10 Hz the real-time budget is 100 ms per frame.

The raw cone scan spans $x \in [0, 7.31]$ m, $y \in [-4.65, 4.99]$ m and $z \in [-0.82, 2.69]$ m. Because $x \geq 0$ everywhere, this is a forward-looking sensor, not a 360° spinning one. x is the forward axis, y lateral and z up. The lowest returns, at $z \approx -0.82$ m, are the ground, so the sensor sits about 0.8 m above the track.

Formula Student cones are small: the standard small cone is roughly 228×228 mm at the base and 325 mm tall. The three best-sampled detected cones have measured heights of 0.311–0.318 m, which matches.

2.3 Classical geometric pipeline versus learning

The pipeline is classical: hand-designed filters, segmentation, model fitting and optimisation, all from PCL [31]. This is the structure of the early Formula Student perception stacks, such as AMZ Driverless [12, 18]: ground removal, clustering, then cone verification. The learning-based alternatives operate on raw points (PointNet [26]), voxels

(VoxelNet [42]) or pillars (PointPillars [19]). They need labelled data, which this task did not provide; with only one labelled scene, a geometric method is the defensible choice. On the camera side, cone detection is usually done with CNN detectors [9]. The image path of the task (ORB features, essential matrix, pose recovery) is standard multi-view geometry [14, 22, 28]; see Section 14. A good general reference for all of the vision background is Szeliski [35].

3 Architecture and coordinate frames

3.1 Pipeline overview

Figure 1 shows the cone pipeline with the measured point and cluster counts at each stage. The same functions run in the GUI (on Compute or slider release) and in the standalone `logic/main.cpp`.

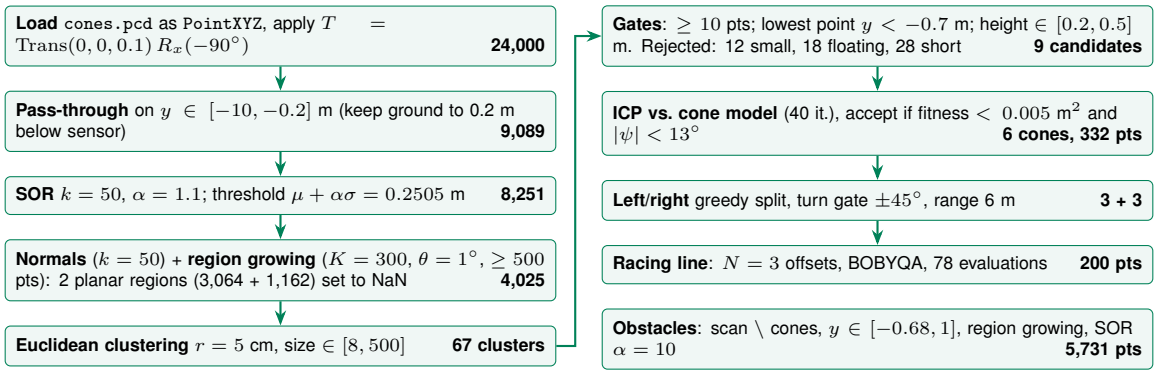


Figure 1: Cone pipeline with GUI default parameters and measured counts. The obstacle branch reuses the full scan and the cone points.

3.2 Homogeneous transforms

A rigid transformation in $SE(3)$ is written as a 4×4 homogeneous matrix

$$T = \begin{bmatrix} R & \boldsymbol{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}, \qquad R \in SO(3),\ \boldsymbol{t} \in \mathbb{R}^3, \qquad T^{-1} = \begin{bmatrix} R^\top & -R^\top \boldsymbol{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}. \tag{1}$$

Points transform as $\tilde{\boldsymbol{p}}' = T\tilde{\boldsymbol{p}}$ with $\tilde{\boldsymbol{p}} = (\boldsymbol{p}, 1)$. The inverse in (1) is used again in odometry (Section 9). The elementary rotations are

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\theta & -s_\theta \\ 0 & s_\theta & c_\theta \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}, \quad R_z(\theta) = \begin{bmatrix} c_\theta & -s_\theta & 0 \\ s_\theta & c_\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

3.3 The levelling transform collapses to one rotation

On load the code builds

```
T = Identity; T.translation() << 0, 0, 0.1;
T.rotate(Rx(pi/2)); T.rotate(Ry(-pi)); T.rotate(Rz(-pi));
```

Eigen's `rotate` post-multiplies (Section 11), so $T = \mathrm{Trans}(0,0,0.1)\,R_x(\frac{\pi}{2})R_y(-\pi)R_z(-\pi)$. Since

$$R_y(-\pi)R_z(-\pi) = \mathrm{diag}(-1,1,-1)\,\mathrm{diag}(-1,-1,1) = \mathrm{diag}(1,-1,-1) = R_x(\pi), \tag{3}$$

the three rotations collapse to $R_x(\frac{\pi}{2})R_x(\pi) = R_x(\frac{3\pi}{2}) = R_x(-\frac{\pi}{2})$:

$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0.1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \Longrightarrow \quad (x', y', z') = (x,\ z,\ -y + 0.1). \tag{4}$$

The printed matrix from the measurement run is exactly this. The odometry program uses the same $R_x(-\frac{\pi}{2})$, so both programs share one convention.

Frame	x	y	z	Used in
Raw sensor	forward, $[0, 7.31]$	lateral	up, ground at -0.82	PCD files
Working (after T)	forward	up, $[-0.82, 2.69]$	lateral, $-y_{\text{raw}} + 0.1$	filters, clustering, ICP
Planner (inside left/right)	lateral ($= z$)	**0**	forward ($= x$)	`distinguishLeftRight`
Racing 2D	lateral	forward	—	`TrackData`

Table 2: Coordinate frames. The README calls x "East" and z "North"; in the data, x is the forward axis and z the lateral axis.

Be careful when presenting this

The report says that in `pcl::PointXYZ` "Z represents north and X east" and that no rotation could fix it. PCL attaches no meaning to the axes. The forward/lateral roles come from the levelling transform above: after $R_x(-90°)$, y is up, x is the sensor's forward axis and z is lateral. The $x \leftrightarrow z$ swap before the left/right step converts this 3D frame into a 2D (lateral, forward) frame. A cleaner answer: "I project to the ground plane by dropping the vertical axis y and use (z, x) as 2D coordinates."

4 Level 1 — Loading and preprocessing

4.1 Pass-through (height crop)

A pass-through filter keeps the points whose chosen field lies in an interval. With y up and the ground at $y \approx -0.82$ m, the band $y \in [-10, -0.2]$ m keeps everything from the ground up to 0.2 m below the sensor, i.e. about 0.62 m above the ground. A 0.325 m cone reaches $y \approx -0.50$ m, so the band keeps whole cones with a 0.3 m margin while removing the upper parts of walls and anything overhead. Measured: 24,000 → 9,089 points (62.1 % removed). Cost: $O(n)$.

4.2 Statistical outlier removal (SOR)

SOR [32] removes isolated points (LiDAR noise, mixed-pixel returns at object edges). For each point $\boldsymbol{p}_i$ with k nearest neighbours $\mathcal{N}_k(i)$:

$$\bar{d}_i = \frac{1}{k} \sum_{j \in \mathcal{N}_k(i)} \|\boldsymbol{p}_i - \boldsymbol{p}_j\|, \qquad \mu = \frac{1}{n} \sum_i \bar{d}_i, \qquad \sigma^2 = \frac{1}{n-1} \sum_i (\bar{d}_i - \mu)^2, \tag{5}$$

and point i is kept if and only if $\bar{d}_i \leq \mu + \alpha\sigma$. PCL implements exactly this, using a $k+1$ query because the point is its own nearest neighbour.

Algorithm 1 Statistical outlier removal (PCL `StatisticalOutlierRemoval`)

Require: cloud P, neighbours k, multiplier α
1: build kd-tree on P ▷ $O(n \log n)$
2: **for** each $\boldsymbol{p}_i \in P$ **do**
3: $\quad \bar{d}_i \leftarrow$ mean distance to k nearest neighbours ▷ $O(k \log n)$ each
4: **end for**
5: $\mu, \sigma \leftarrow$ mean and sample std. of $\{\bar{d}_i\}$
6: **return** $\{\boldsymbol{p}_i : \bar{d}_i \leq \mu + \alpha\sigma\}$

Numbers and justification. On the 9,089 cropped points (measured): $\mu = 0.1422$ m, $\sigma = 0.0985$ m, so the threshold is $0.1422 + 1.1 \times 0.0985 = 0.2505$ m. The filter removed 838 points (9.22 %), leaving 8,251. If the $\bar{d}_i$ were Gaussian, a one-sided cut at 1.1σ would remove $1 - \Phi(1.1) = 13.6\,\%$. Fewer points are removed because the distribution of mean distances is right-skewed: a heavy tail of isolated points inflates σ. The value $\alpha = 1.1$ was tuned interactively with the GUI slider (range 0.5–2.0). Smaller values start eating the sparse far cones; larger values let wall noise survive and create spurious clusters.

Numbers to remember

24,000 → 9,089 (crop) → 8,251 (SOR, $k = 50$, $\alpha = 1.1$, threshold 0.2505 m, 9.2 % removed).

5 Level 2 — Segmentation: removing walls and clustering

5.1 Surface normals by PCA

Region growing needs a normal and a curvature at every point. For the $k = 50$ neighbours of $\boldsymbol{p}$, with centroid $\bar{\boldsymbol{p}}$, form the covariance

$$C = \frac{1}{k}\sum_{j=1}^{k}(\boldsymbol{p}_j - \bar{\boldsymbol{p}})(\boldsymbol{p}_j - \bar{\boldsymbol{p}})^\top = \sum_{l=0}^{2}\lambda_l\,\boldsymbol{v}_l\boldsymbol{v}_l^\top, \qquad 0 \le \lambda_0 \le \lambda_1 \le \lambda_2. \tag{6}$$

The normal is the eigenvector of the smallest eigenvalue, $\boldsymbol{n} = \boldsymbol{v}_0$ (the direction of least spread [16]). The surface variation is

$$\sigma(\boldsymbol{p}) = \frac{\lambda_0}{\lambda_0 + \lambda_1 + \lambda_2} \in \left[0, \tfrac{1}{3}\right], \tag{7}$$

which is 0 on a plane and $\frac{1}{3}$ for isotropic scatter [23]. PCL calls it "curvature".

5.2 Region growing

Region growing [27] merges neighbouring points whose normals are nearly parallel. In PCL's version, points are sorted by curvature and the lowest-curvature point seeds a region. A neighbour $\boldsymbol{q}$ of the current seed $\boldsymbol{s}$ joins the region if

$$|\boldsymbol{n_s} \cdot \boldsymbol{n_q}| \;\ge\; \cos\theta_{\text{th}}, \tag{8}$$

and becomes a new seed if $\sigma(\boldsymbol{q}) < c_{\text{th}}$. The absolute value makes the test independent of the sign ambiguity of PCA normals.

Algorithm 2 Region growing (PCL `RegionGrowing`), as used to remove walls

Require: cloud, normals, curvatures; K neighbours; θ_{th}; c_{th}; min size m

```
sort points by curvature (ascending); mark all unlabelled
while an unlabelled point exists do
    s ← unlabelled point with lowest curvature; region A ← {s}; seeds ← [s]
    while seeds not empty do
        pop s; for each unlabelled q ∈ N_K(s)
            if |n_s · n_q| ≥ cos θ_th then add q to A; if σ(q) < c_th push q to seeds
    end while
    keep A only if |A| ≥ m
end while
```

Parameters and why they isolate walls. $\theta_{\text{th}} = 1^\circ$, $K = 300$, $c_{\text{th}} = 1$, $m = 500$. A 1° tolerance only lets near-perfect planes grow. A cone is curved and has at most a few hundred points (113 for the best-sampled one), so it can never form a 500-point region. Walls and ground are large and flat, so they do. Because $\sigma \le \frac{1}{3} < 1$, the curvature test $c_{\text{th}} = 1$ always passes: every added point becomes a seed, and only the normal-angle test acts. The large neighbourhood $K = 300$ lets regions jump small gaps in sparse wall areas.

Measured: 2 regions of 3,064 and 1,162 points (4,226 points, 51.2 % of the cloud) are removed, leaving 4,025 finite points. Time: normals 20.1 ± 0.5 ms, region growing 180 ± 21 ms. Region growing is the most expensive stage of the whole cone pipeline.

Why region growing and not RANSAC planes? RANSAC [11] fits one plane per run and needs to know how many planes to remove. Here the number of walls depends on the scene. An early attempt of the project, "remove the 3 biggest Euclidean clusters", needed exactly this prior. Region growing finds all large smooth regions in one pass, with no count to specify.

5.3 Euclidean cluster extraction

The remaining points are grouped into objects by Euclidean clustering [30]. Clusters are the connected components of the graph G_r with an edge between $\boldsymbol{p}_i$ and $\boldsymbol{p}_j$ whenever $\|\boldsymbol{p}_i - \boldsymbol{p}_j\| \le r$. They are found by breadth-first search with kd-tree radius queries [4, 21], and kept if their size is in $[m_{\min}, m_{\max}]$. This is DBSCAN [10] with minPts $= 1$ (single-linkage clustering cut at distance r). K-means would need the number of clusters in advance and assumes convex, similar-sized clusters, so it is a poor fit for an unknown number of small objects.

Parameters. $r = 0.05$ m, size $\in [8, 500]$. The 5 cm tolerance sits between the point spacing on a cone at 3–5 m (a few cm) and the gap between separate objects (tens of cm). The upper bound of 500 discards any surviving wall fragment. Measured: 67 clusters (sizes 8–482) holding 2,397 points, in 6.4 ± 2.0 ms.

A subtle detail: NaNs instead of deleted points

Walls are removed with `ExtractIndices` using `setNegative(true)` and `setKeepOrganized(true)`: removed points are overwritten with NaN, not erased. The cloud therefore still has 8,251 entries but only 4,025 finite points. This keeps the indices of later regions valid while removing earlier ones (Section 11). The kd-tree used by clustering skips non-finite points.

6 Level 3 — Classifying cones and obstacles

6.1 Cheap geometric gates first

Before any ICP, each cluster passes three $O(|\text{cluster}|)$ tests (the bounding box is computed in a single pass):

1. at least 10 points — 12 clusters fail;
2. the lowest point satisfies $y_{\min} < -0.7$ m, i.e. the object touches the ground (ground at -0.82) — 18 floating fragments fail;
3. the height $y_{\max} - y_{\min} \in [0.2, 0.5]$ m — 28 clusters are too short.

This leaves 9 candidates. The gates are justified by cone physics: a cone stands on the ground and is ≈0.33 m tall. Partial scans of far cones lose the top, which is why the lower bound is 0.2 rather than 0.3. Running cheap gates before an expensive verifier is the classic cascade design.

6.2 The ideal cone model

`GenConeTool/gen_cone.py` samples the *lateral surface* of a cone of height h and base radius ρ (no base, no interior), which is what a LiDAR sees:

$$u, v \sim \mathcal{U}(0,1), \quad \eta = u, \quad r = \rho(1-\eta), \quad \phi = 2\pi v, \qquad \boldsymbol{p} = (r\cos\phi,\ \eta h,\ r\sin\phi). \tag{9}$$

The shipped `ideal_cone.ply` has 150 points, $h = 0.40$ m and $\rho = 0.20$ m (measured extents: $y \in [0.003, 0.398]$, max radius 0.198).

A sampling subtlety worth knowing

Drawing the height uniformly does not give uniform density over the surface. The area element of a cone grows linearly with the distance from the apex, so area-uniform sampling needs $\eta = 1 - \sqrt{u}$, which puts the centroid at $h/3$ above the base. The model's centroid is at $y = 0.2016 \approx h/2$ (measured), i.e. the apex region is over-represented. Because both the model and each cluster are centred on their own centroids before ICP, this shifts the initial vertical alignment, and ICP has to correct it. Accepted fits still reach 2–3 cm RMS, so the effect is small, but it is an easy improvement.

6.3 ICP alignment as a shape test

For each candidate, the cluster P (source) and the model Q (target) are both translated to their centroids. Then point-to-point ICP [5] is run for at most 40 iterations, with transformation and fitness epsilons of 10^{-8}. The ICP algorithm itself is given in Section 9.1. Two outputs are used.

Fitness. PCL's `getFitnessScore()` is the mean squared nearest-neighbour distance from the aligned source to the target:

$$F = \frac{1}{|P|} \sum_{\boldsymbol{p} \in P} \min_{\boldsymbol{q} \in Q} \|T\boldsymbol{p} - \boldsymbol{q}\|^2 \quad [\text{m}^2], \qquad \text{RMS} = \sqrt{F}. \tag{10}$$

The threshold $F < 0.005$ m^2 therefore means an RMS residual below $\sqrt{0.005} = 7.1$ cm. The score is asymmetric: it measures how well the cluster lies on the model, not the reverse. A model that is larger than the real cone (0.40 m vs. 0.325 m) is therefore tolerated, because a partial cluster can lie on any band of the model surface.

Yaw. From the rotation block R of the final transform, the code computes

$$\psi = \operatorname{atan2}(r_{13},\ r_{33}), \tag{11}$$

which is exactly θ for a pure $R_y(\theta)$, i.e. the rotation about the vertical axis y (heading in a Y-X-Z decomposition). The cluster is accepted if $F < 0.005$ **and** $|\psi| < 13°$.

Algorithm 3 Cone classification (`processClusterForConeDetection`)

Require: clusters $\{P_c\}$, model Q (centred), thresholds $F_{\max} = 0.005$, $\psi_{\max} = 13°$

for each cluster P_c **do**
  if $|P_c| < 10$ **or** $y_{\min} > -0.7$ **or** $h \notin [0.2, 0.5]$ **then continue**
  end if
  $P_c \leftarrow P_c - \bar{\boldsymbol{p}}_c$ ▷ centre the cluster
  $(T, F) \leftarrow \mathrm{ICP}(P_c \to Q,\ 40 \text{ iterations})$
  $\psi \leftarrow \mathrm{atan2}(T_{13}, T_{33})$
  if converged **and** $F < F_{\max}$ **and** $|\psi| < \psi_{\max}$ **then** mark c as cone
  end if
end for
cone centre $\leftarrow$ mean of the original (uncentred) cluster points

6.4 Results for the nine candidates

Cluster	Points	$y_{\min}$ [m]	Height [m]	Fitness [m^2]	RMS [cm]	$\lvert\psi\rvert$ [°]	Decision
C2	126	-0.764	0.328	0.0009	3.0	45.27	reject (yaw)
C4	113	-0.752	0.317	0.0011	3.3	12.31	**cone**
C5	69	-0.799	0.215	0.0027	5.2	17.49	reject (yaw)
C6	67	-0.755	0.311	0.0007	2.6	1.23	**cone**
C7	64	-0.766	0.318	0.0010	3.2	3.25	**cone**
C9	55	-0.718	0.258	0.0009	3.0	22.77	reject (yaw)
C15	34	-0.781	0.211	0.0007	2.6	3.43	**cone**
C16	32	-0.762	0.301	0.0006	2.4	12.50	**cone**
C29	22	-0.772	0.241	0.0004	2.0	8.03	**cone**

Table 3: Measured ICP results for the nine clusters that pass the geometric gates (cluster ids follow PCL's size-descending order).

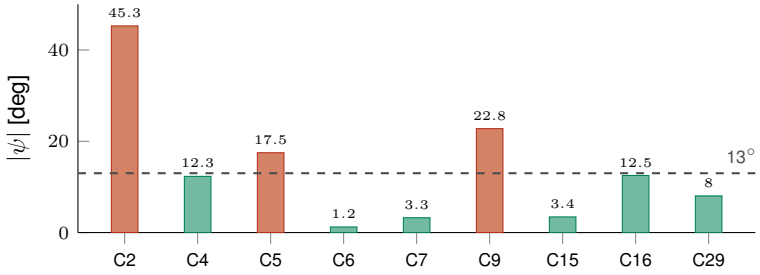


Figure 2: Yaw required by ICP for each candidate. Green: accepted; red: rejected. All nine pass the fitness test, so the yaw gate is what separates them.

What actually decides cone vs. non-cone

All nine candidates have fitness $\leq 0.0027 < 0.005$, so on this scene the fitness threshold rejects nothing. Any threshold between 0.0027 and 0.9 (the slider maximum) gives the same 6 cones, and any threshold below 0.0004 gives none. The real discriminators are the geometric gates (46 rejections) and the yaw gate (3 rejections). The yaw threshold of 13° lies in the gap between the largest accepted value (12.50°) and the smallest rejected one (17.49°). It is only 0.5° above an accepted cone; the midpoint of the gap, 15°, would give the largest margin on both sides. Say this openly: it shows you understand the sensitivity.

Why can a yaw test work for a rotationally symmetric object? For a perfect cone, rotation about its own axis does not change the residual: yaw is a null direction of the ICP cost. A real upright cone, already centred, starts with identity rotation. ICP is a local descent method, so it has no reason to move along a direction that does not

lower the cost, and the yaw stays small (1.2–12.5° here; the non-zero values come from the random, non-symmetric sampling of the model and the one-sided view). A non-cone fragment, such as the piece of the north wall mentioned in the report, has to be rotated substantially to be draped over the model surface, so it ends with a large rotation (17–45°). The gate is an empirical heuristic, not a theorem. A more principled feature would be the tilt of the fitted axis, $\arccos\left((R\hat{\boldsymbol{y}})\cdot\hat{\boldsymbol{y}}\right)$, combined with the yaw.

6.5 Point density versus range

The six accepted cones have 113, 67, 64, 34, 32 and 22 points at planar ranges 2.90, 2.96, 4.00, 5.37, 4.95 and 4.68 m. For a sensor with fixed angular resolution, the number of returns on an object of fixed size scales as the solid angle it subtends, $n \propto r^{-2}$. A least-squares fit of $\log n$ on $\log r$ over the six cones gives an exponent of -1.96 (Figure 3), close to the theory. The scatter (e.g. 67 points at 2.96 m) comes from partial occlusion, orientation and the SOR step. This also explains why far cones are hard: a cone at 6–7 m yields only about 15 points.

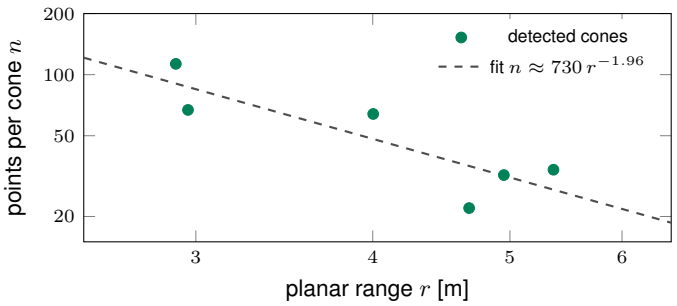


Figure 3: Points per detected cone versus range (measured).

6.6 Cone centres

The cone position is the centroid of the original cluster points. Measured centres (working frame: x forward, z lateral) are A $(2.849, -0.562)$, B $(2.879, 0.695)$, C $(3.977, 0.457)$, D $(5.328, 0.649)$, E $(4.882, -0.827)$, F $(4.511, 1.243)$ as (x, z) in metres, with y between -0.70 and -0.61. A one-sided view biases the centroid towards the sensor by a few cm. Fitting the model's apex axis and using its ground point would remove this bias.

6.7 Obstacles

`get_obstacle_cloud` returns "everything that is not a cone and not a wall":

1. set difference: `SegmentDifferences` removes from the scan every point with a cone point within a squared distance of 10^{-6} m^2 (1 mm), i.e. an exact match;
2. pass-through $y \in [-0.68, 1]$ m, which removes the ground and keeps objects standing above it;
3. the same region growing as before, to remove walls;
4. a very permissive SOR ($k = 50$, $\alpha = 10$) that only removes extreme isolated points.

Measured: 5,731 finite obstacle points (organised size 10,295), 267 ± 21 ms.

7 Level 4a — Separating left and right edges

There are no cone colours (intensity is not loaded), so the two edges must be inferred from geometry alone. Two greedy approaches were prototyped in MATLAB:

1. relative-angle competitive allocation (implemented in C++): grow two chains from the two cones closest to the car, each taking its nearest unassigned cone, with a turning-angle gate;
2. simulated robot motion (`GreedyLoop.m`): a virtual car moves forward, scores every (left, right) pair in its field of view by distance, heading alignment and corridor width, and steers towards the best midpoint. It is more general, but it needs dense cones.

7.1 The turning angle

For three consecutive points $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$ in a chain,

$$\delta = \operatorname{wrap}\left(\operatorname{atan2}(c_y - b_y,\, c_x - b_x) - \operatorname{atan2}(b_y - a_y,\, b_x - a_x)\right)\cdot\tfrac{180}{\pi}, \qquad \operatorname{wrap}(\alpha) \in (-\pi, \pi], \tag{12}$$

where wrap adds or subtracts 2π until the angle lies in $(-\pi, \pi]$. A chain may only turn by $\delta \in [-45°, 45°]$ per step.

Algorithm 4 Greedy left/right split as implemented (`distinguishLeftRight`)

Require: cone centres, robot position $(0, 0)$, gate $[-45°, 45°]$, range $d_{\max} = 6$ m

map each centre to the planner frame $(x, y) \leftarrow (z,\ 0)$ ▷ forward coordinate is not used
keep cones with $\|(x, y)\| \le d_{\max}$; argsort by distance to the robot
$\mathcal{L} \leftarrow$ [closest], $\mathcal{R} \leftarrow$ [second closest]
for at most $N - 2$ rounds **do**
    if one cone remains **then** append it to the side with smaller $|\delta|$; **break**
    end if
    $\ell \leftarrow$ nearest unassigned to $\mathcal{L}_{\text{last}}$; $r \leftarrow$ nearest unassigned to $\mathcal{R}_{\text{last}}$
    if $\ell = r$ **then** the closer side keeps it; the other side takes its second-nearest
    end if
    $\delta_L, \delta_R \leftarrow$ turning angles (0 if a chain has fewer than 2 cones)
    if δ_L or δ_R outside the gate **then** swap $(\ell, \delta_L) \leftrightarrow (r, \delta_R)$; drop any candidate still outside
    end if
    append the surviving candidates and mark them assigned
end for

Complexity: each round scans the unassigned cones, so the cost is $O(N^2)$ for N cones, negligible for $N = 6$ (measured < 0.2 ms). Measured output: left $=$ [C, D, F], right $=$ [A, E, B] (3 + 3).

The left/right split works on one coordinate only

The code maps `p.x = src.z; p.z = src.x; p.y = 0;` and then measures every distance and angle with `p.x` and `p.y`. The forward coordinate, stored in `p.z`, is never used, so the split sees only lateral positions:

- distances are $|z_i - z_j|$, and the 6 m range limit is a lateral limit;
- every turning angle is $0°$ or $180°$, because $\operatorname{atan2}(0, \cdot) \in \{0, \pi\}$;
- the right chain A $\to$ E $\to$ B goes forward 2 m and then back 2 m.

The README's description ("left if the cross product with the heading is > 0") does not match the code either. The MATLAB prototype is correct in 2D; the bug was introduced in the C++ port. Setting `p.y = src.x` fixes the frame, but on this sharp bend the $\pm 45°$ gate then drops cones and only 2 gates remain (checked by experiment). A robust replacement is Delaunay triangulation of the cones with midpoint-graph search, as used by AMZ [18], or cone colour recovered from LiDAR intensity [12].

8 Level 4b — Racing line by constrained optimisation

8.1 Formulation

Pair the i-th left and right cones, $\boldsymbol{l}_i, \boldsymbol{r}_i$, for $i = 0, \dots, N-1$ with $N = \min(|\mathcal{L}|, |\mathcal{R}|)$. Define

$$\boldsymbol{m}_i = \tfrac{1}{2}(\boldsymbol{l}_i + \boldsymbol{r}_i), \qquad w_i = \tfrac{1}{2}\,\|\boldsymbol{r}_i - \boldsymbol{l}_i\|\,, \tag{13}$$

$$\boldsymbol{\tau}_i = \begin{cases} \boldsymbol{m}_1 - \boldsymbol{m}_0 & i = 0 \\ \boldsymbol{m}_{i+1} - \boldsymbol{m}_{i-1} & 0 < i < N-1 \\ \boldsymbol{m}_{N-1} - \boldsymbol{m}_{N-2} & i = N-1 \end{cases}, \qquad \boldsymbol{n}_i = \frac{1}{\|\boldsymbol{\tau}_i\|}\begin{bmatrix} -\tau_{i,y} \\ \tau_{i,x} \end{bmatrix}, \tag{14}$$

i.e. midpoints, half-widths, central-difference tangents and left-pointing unit normals (tangent rotated by $+90°$; the norm is clamped at 10^{-12}). The decision variables are the lateral offsets $\boldsymbol{d} \in \mathbb{R}^N$, which place the racing points at $\boldsymbol{p}_i(\boldsymbol{d}) = \boldsymbol{m}_i + d_i \boldsymbol{n}_i$. The problem is

$$\begin{aligned} \min_{\boldsymbol{d} \in \mathbb{R}^N} \quad & J(\boldsymbol{d}) = \sum_{i=1}^{N-2} \kappa_i(\boldsymbol{d})^2 + \lambda \sum_{i=0}^{N-2} (d_{i+1} - d_i)^2, \qquad \lambda = 0.1 \\ \text{s.t.} \quad & -w_i \le d_i \le w_i, \quad i = 0, \dots, N-1, \end{aligned} \tag{15}$$

with the discrete curvature

$$\kappa_i = \frac{\boldsymbol{a}_i \times \boldsymbol{b}_i}{\left(\|\boldsymbol{a}_i\|^2 + \varepsilon\right)^{3/2}}, \qquad \boldsymbol{a}_i = \boldsymbol{p}_i - \boldsymbol{p}_{i-1},\ \boldsymbol{b}_i = \boldsymbol{p}_{i+1} - \boldsymbol{p}_i,\ \varepsilon = 10^{-6}, \tag{16}$$

where $\boldsymbol{a} \times \boldsymbol{b} = a_x b_y - a_y b_x$ is the scalar 2D cross product. This is the same approach as minimum-curvature racing lines in the literature [15], with a box-constrained offset parametrisation: the box keeps the line inside the track by construction.

8.2 Where the curvature formula comes from

For a curve $\boldsymbol{r}(s) = (x(s), y(s))$ the signed curvature is $\kappa = \dfrac{x'y'' - y'x''}{(x'^2 + y'^2)^{3/2}}$. With unit parameter step h, backward differences give $\boldsymbol{r}' \approx \boldsymbol{a}/h$ and $\boldsymbol{r}'' \approx (\boldsymbol{b} - \boldsymbol{a})/h^2$. Then

$$\kappa \approx \frac{\boldsymbol{a} \times (\boldsymbol{b} - \boldsymbol{a})/h^3}{\|\boldsymbol{a}\|^3 / h^3} = \frac{\boldsymbol{a} \times \boldsymbol{b}}{\|\boldsymbol{a}\|^3}, \tag{17}$$

because $\boldsymbol{a} \times \boldsymbol{a} = 0$ and the h^3 factors cancel. This is (16). The ε stops the division from blowing up when two consecutive points coincide (e.g. when a left and a right cone are paired at the same spot). Writing $\boldsymbol{a} \times \boldsymbol{b} = \|\boldsymbol{a}\| \, \|\boldsymbol{b}\| \sin\theta$ with θ the turning angle gives

$$\kappa_i = \frac{\|\boldsymbol{b}\|}{\|\boldsymbol{a}\|} \cdot \frac{\sin\theta}{\|\boldsymbol{a}\|} \xrightarrow{\|\boldsymbol{a}\| = \|\boldsymbol{b}\| = s} \frac{\sin\theta}{s}, \qquad \text{while} \qquad \kappa^{\text{Menger}} = \frac{2\sin\theta}{\|\boldsymbol{p}_{i+1} - \boldsymbol{p}_{i-1}\|} = \frac{2\sin(\theta/2)}{s}. \tag{18}$$

The two agree for small θ but differ for large ones (Figure 4).

Why $\lambda = 0.1$? The smoothness term is a first-difference Tikhonov regulariser. It discourages zig-zag offsets that the curvature term barely penalises, and it makes the problem better conditioned when few curvature terms exist (with $N = 3$ there is only one). With $\lambda = 0.1$ it is about an order of magnitude below the curvature term at the start ($J(\mathbf{0}) = 0.129$ here), so curvature dominates.

8.3 BOBYQA

BOBYQA (Bound Optimisation BY Quadratic Approximation) [25] is a derivative-free trust-region method [8]. At iterate $\boldsymbol{x}_k$ it keeps a quadratic model Q_k that interpolates J at m points (Powell recommends $m = 2n + 1$, NLopt's choice). It then solves

$$\min_{\boldsymbol{s}} \; Q_k(\boldsymbol{x}_k + \boldsymbol{s}) \quad \text{s.t.} \quad \|\boldsymbol{s}\| \le \Delta_k, \;\; \boldsymbol{\ell} \le \boldsymbol{x}_k + \boldsymbol{s} \le \boldsymbol{u}, \tag{19}$$

evaluates J once at the trial point, and accepts it and resizes the trust region according to the ratio of actual to predicted decrease, $\rho_k = \frac{J(\boldsymbol{x}_k) - J(\boldsymbol{x}_k + \boldsymbol{s})}{Q_k(\boldsymbol{x}_k) - Q_k(\boldsymbol{x}_k + \boldsymbol{s})}$. The model's Hessian is updated by the least-change (minimum Frobenius norm) rule, so only one function value per iteration is needed. Bounds are handled natively, which is exactly what (15) needs.

Why derivative-free? No gradient is coded: the callback sets `grad` to zero, and `LN_` algorithms never request it. With $N \le$ a few tens of variables, a function evaluation costs microseconds (measured: the whole optimisation takes about 0.1 ms), so there is no speed reason to derive gradients. A gradient-based alternative, IPOPT [39] via CasADi [1] with automatic differentiation, was prototyped in Python and dropped because the C++ API documentation was lacking.

Settings. Initial guess $\boldsymbol{d} = \mathbf{0}$ (centreline); `xtol_rel` $= 10^{-6}$; `maxeval` $= 1000$.

Algorithm 5 Racing line (`get_racing_line_point_cloud`)

Require: left and right chains (2D: lateral, forward)
1: build $\boldsymbol{m}_i, w_i, \boldsymbol{n}_i$ ▷ `TrackData` constructor
2: BOBYQA on (15) with bounds $\pm w_i$, start $\boldsymbol{d} = \mathbf{0}$
3: **if** NLopt result < 0 **then return** empty cloud
4: **end if**
5: draw 100 points from the origin to $\boldsymbol{p}_0$
6: **for** $i = 1$ to $N - 2$ **do** draw 100 points from $\boldsymbol{m}_{i-1} + d_i \boldsymbol{n}_{i-1}$ to $\boldsymbol{m}_i + d_i \boldsymbol{n}_i$
7: **end for**
8: swap back to the working frame (lateral $\to z$, forward $\to x$), height $y = 0$

8.4 Measured results on the scene

i	$\boldsymbol{m}_i$ (lat, fwd) [m]	w_i [m]	$\boldsymbol{n}_i$	$d_i^\star$ [m]	$\boldsymbol{p}_i^\star$ (lat, fwd) [m]
0	$(-0.052,\ 3.413)$	0.760	$(-1.000, -0.022)$	-0.554	$(0.502,\ 3.425)$
1	$(-0.089,\ 5.105)$	0.771	$(-0.266,\ 0.964)$	-0.554	$(0.058,\ 4.570)$
2	$(\ 0.969,\ 3.695)$	0.861	$(\ 0.800,\ 0.601)$	-0.554	$(0.526,\ 3.362)$

Table 4: Racing-line optimisation on the detected cones (measured): $J(\mathbf{0}) = 0.1289$, $J^\star < 5 \times 10^{-5}$, NLopt code 4 (`XTOL_REACHED`), 78 evaluations (identical over 10 runs), output cloud of 200 points.

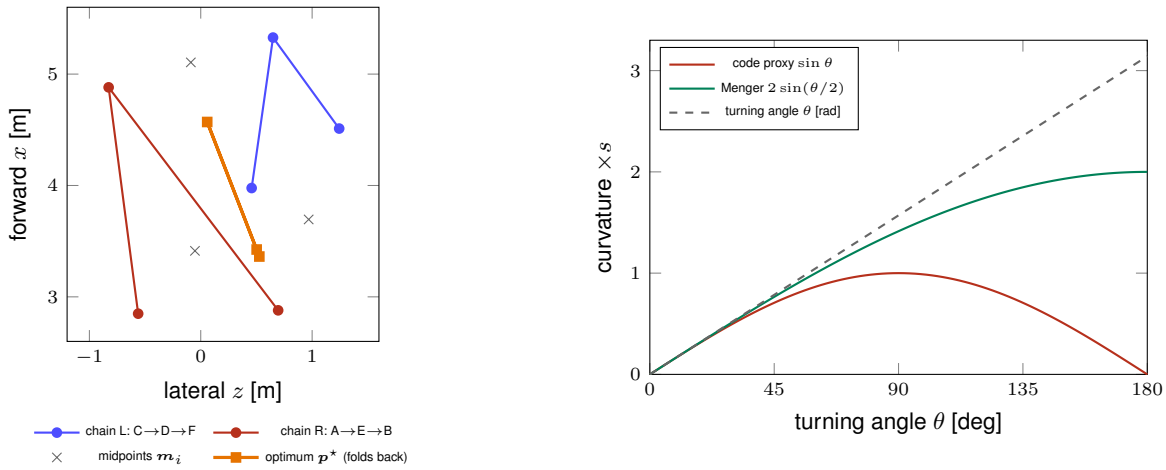


Figure 4: Left: measured split and racing line (planner frame; the sensor is at the origin, below the plotted area). Right: the curvature proxy of (16) for equal segment lengths falls back to zero at 180°, so a full reversal costs nothing.

The optimal racing line is degenerate on this scene

All three offsets come out equal (-0.554 m), and $\boldsymbol{p}_0^\star$ and $\boldsymbol{p}_2^\star$ are 7 cm apart: the line goes out and comes straight back, with a turning angle of 180°. Its cost is zero because $\boldsymbol{a} \times \boldsymbol{b} = 0$ for antiparallel vectors. The proxy measures $\sin\theta$, which cannot tell "straight" from "U-turn". Two causes combine: (i) the midpoint sequence already folds back (midpoint 2 is behind midpoint 1) because of the left/right ordering above; (ii) the objective rewards the fold. The GUI hides this, because the drawing loop never draws the last point (Algorithm 5 stops at $i = N - 2$) and reuses d_i for both ends of a segment. **Fixes:** use the signed turning angle $\operatorname{atan2}(\boldsymbol{a} \times \boldsymbol{b},\ \boldsymbol{a} \cdot \boldsymbol{b})$ or the Menger curvature; order the gates along the track first; draw $\boldsymbol{p}_{i-1}^\star \to \boldsymbol{p}_i^\star$ for $i = 1, \ldots, N-1$.

8.5 Validation on a toy track: NLopt versus CasADi/IPOPT

The same objective was first written in Python with CasADi/IPOPT and then ported to C++/NLopt. On the toy track $\mathcal{L} = \{(0,0), (0,3), (2,5), (3,5)\}$, $\mathcal{R} = \{(0,0), (1,2), (3,4), (4,4)\}$ (stored in `comparison.txt`), the two solvers give the racing points in Table 5. Evaluating (15) at both solutions shows that BOBYQA found the lower objective.

Solver	**Racing points**	**Offsets $\boldsymbol{d}^\star$**	$J(\boldsymbol{d}^\star)$
Centreline ($\boldsymbol{d} = \mathbf{0}$)	$(0,0), (0.5, 2.5), (2.5, 4.5), (3.5, 4.5)$	$(0, 0, 0, 0)$	0.06607
CasADi + IPOPT	$(0,0), (0.708, 2.384), (2.500, 4.500), (3.500, 4.578)$	$(0, -0.238, 0, 0.078)$	0.05305
NLopt BOBYQA	$(0,0), (0.766, 2.352), (2.570, 4.395), (3.500, 4.452)$	$(0, -0.304, -0.127, -0.048)$	**0.05223**

Table 5: Toy-track comparison. Point deviations between the solvers are at most 0.127 m. Both improve on the centreline (by 19.7 % and 20.9 %).

The objective is non-convex, because the curvature is a rational function of $\boldsymbol{d}$, so the two solvers can stop at different stationary points. Here BOBYQA's point is 1.5 % better. Note also $w_0 = 0$ (the first left and right cones coincide), so d_0 is pinned to 0 by the bounds.

9 Level 5 — LiDAR odometry with ICP

9.1 ICP

Given a source $P = \{\boldsymbol{p}_i\}$ and a target Q, point-to-point ICP [5] alternates between correspondence and alignment:

$$c(i) = \arg\min_j \|T_k \boldsymbol{p}_i - \boldsymbol{q}_j\|, \qquad (R, \boldsymbol{t}) = \arg\min_{R \in SO(3),\, \boldsymbol{t}} \sum_i \left\| R\boldsymbol{p}_i + \boldsymbol{t} - \boldsymbol{q}_{c(i)} \right\|^2. \tag{20}$$

The alignment step has a closed form [2, 17, 37]. With centroids $\bar{\boldsymbol{p}}, \bar{\boldsymbol{q}}$ of the matched pairs,

$$H = \sum_i (\boldsymbol{p}_i - \bar{\boldsymbol{p}})(\boldsymbol{q}_{c(i)} - \bar{\boldsymbol{q}})^\top = U \Sigma V^\top, \qquad R = V \operatorname{diag}\big(1, 1, \det(VU^\top)\big)\, U^\top, \qquad \boldsymbol{t} = \bar{\boldsymbol{q}} - R\bar{\boldsymbol{p}}. \tag{21}$$

The $\det$ term prevents a reflection. Each iteration does not increase the mean squared error, so ICP converges monotonically, but only to a local minimum [5]. A good initial guess and a bounded correspondence distance are therefore essential. Variants: point-to-plane [7] minimises $\sum_i \big((R\boldsymbol{p}_i + \boldsymbol{t} - \boldsymbol{q}_{c(i)}) \cdot \boldsymbol{n}_{c(i)}\big)^2$ and converges faster on structured scenes. GICP [33] uses local covariances on both clouds. NDT [6] replaces points by Gaussians. Surveys and modern practice: Pomerleau et al. [24], Rusinkiewicz and Levoy [29], Vizzo et al. [38], Zhang and Singh [40].

Algorithm 6 Point-to-point ICP (PCL `IterativeClosestPoint`)

Require: source P, target Q, $T_0 = I$, max iterations $k_{\max}$, max correspondence distance $d_{\max}$

1: build kd-tree on Q
2: **for** $k = 1$ to $k_{\max}$ **do**
3: match each $T\boldsymbol{p}_i$ to its nearest $\boldsymbol{q}_{c(i)}$; drop pairs farther than $d_{\max}$ ▷ $O(n \log m)$
4: $(R, \boldsymbol{t}) \leftarrow$ SVD closed form; $T \leftarrow (R, \boldsymbol{t})\, T$
5: **if** ΔT below epsilon **or** MSE change below epsilon **then break**
6: **end if**
7: **end for**
8: **return** T, fitness F

9.2 Preprocessing (measured counts)

1. Pass-through on the raw forward axis, $x \in [-1, 10]$ m: 24,000 $\rightarrow$ 23,666 and 23,740 points. The raw scans reach 36.7 m, but distant returns are sparse and change most between frames.
2. Levelling: $T_1 = R_x(-90°)R_y(-10°)$ and $T_2 = R_x(-90°)R_y(-3°)$. The R_y term is a pitch correction in the raw frame, hand-tuned per scan so that the ground is horizontal and the remaining motion is mostly planar.
3. SOR with $k = 50$, $\alpha = 1.0$: $\rightarrow$ 21,508 and 21,239 points (9.1 % and 10.5 % removed; a Gaussian would predict $1 - \Phi(1) = 15.9\,\%$).

ICP is then run with `first` as source and `second` as target, for at most 70 iterations, with all other settings at PCL defaults.

9.3 Result

$$T = \begin{bmatrix} 0.9957 & 0.0930 & 0.0063 & -0.5187 \\ -0.0920 & 0.9914 & -0.0935 & 0.2486 \\ -0.0149 & 0.0925 & 0.9956 & 0.2608 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad F = 0.49974 \text{ m}^2 \text{ (RMS } 0.707 \text{ m)}. \tag{22}$$

The rotation angle is $\theta = \arccos\big((\operatorname{tr} R - 1)/2\big) = 7.56°$ about the axis $(0.707, 0.081, -0.703)$. Its component about the vertical axis y (yaw) is only $0.36°$; the rest is tilt. $\det R = 1.00001$, so R is a valid rotation to numerical precision.

9.4 From the transform to the vehicle displacement

PCL's convention is that the final transform maps the source into the target: $\boldsymbol{p}^{(2)} \approx T\boldsymbol{p}^{(1)}$, i.e. $T = T_{21}$ maps frame-1 coordinates to frame-2 coordinates. The pose of the vehicle at time 2, expressed in frame 1, is the inverse (1):

$$T_{12} = T_{21}^{-1} = \begin{bmatrix} R^\top & -R^\top \boldsymbol{t} \\ \boldsymbol{0}^\top & 1 \end{bmatrix} \quad \Longrightarrow \quad \Delta\boldsymbol{x} = -R^\top \boldsymbol{t} = (+0.543,\ -0.222,\ -0.233) \text{ m } \ (x, y, z). \tag{23}$$

So the vehicle moved **0.543 m forward** (x) and **0.233 m laterally** ($-z$), **0.591 m in the plane**. A sanity check confirms the sign: $t_x = -0.519$ means static objects appear about 0.5 m closer in the second scan, which happens when the vehicle drives forward. If the scans were consecutive frames at 10 Hz, 0.59 m per frame would mean about 5.9 m/s (≈21 km/h), which is plausible. The frame rate is an assumption, not given in the data.

What the program prints, and why it differs

The program prints "Final transformation (second → first)" and then **North 0.2331, East** -0.5432. In PCL the matrix maps first → second (the label is reversed). `computeNorthEast(T, true, true)` first inverts ($-R^\top \boldsymbol{t}$) and then negates again, so it prints $+R^\top \boldsymbol{t} = (-0.543,\ 0.222,\ 0.233)$, i.e. minus the physical displacement. "East" is the x (forward) axis and "North" the z (lateral) axis. The magnitude is robust: $\|\boldsymbol{t}\|_{xz} = 0.581$ m and $\|R^\top \boldsymbol{t}\|_{xz} = 0.591$ m differ by only 1 cm because $R \approx I$. Present the result as "≈0.54 m forward and ≈0.23 m lateral". The side-by-side viewer is also mislabelled: its right panel overlays `first` (white) with T·`first` (red), not with `second`.

9.5 How good is this estimate?

There is no ground truth, so the quality has to be argued from internal evidence. Table 6 gives the fraction of source points whose nearest target point lies within a given distance, before and after ICP.

	< 5 cm	< 10 cm	< 20 cm	< 50 cm	median NN	MSE [m^2]
Before ICP (identity)	12.7 %	21.8 %	32.7 %	55.0 %	0.417 m	0.6405
After project ICP	4.1 %	10.8 %	24.9 %	53.9 %	0.453 m	0.4997

Table 6: Nearest-neighbour statistics of the 21,508 source points (measured).

ICP stopped at the iteration cap, and close inliers got worse

- The convergence state is `CONVERGENCE_CRITERIA_ITERATIONS`: ICP used all 70 iterations. In PCL, `hasConverged()` returns `true` in this case too (unless `failure_after_max_iter` is set), so "ICP converged" in the log does not mean the tolerances were met.
- The maximum correspondence distance is PCL's default $\sqrt{\texttt{DBL_MAX}} \approx 1.3 \times 10^{154}$, i.e. unbounded. Every point is matched, including points in regions seen in only one scan. Those far pairs dominate the sum of squares, so ICP lowers the mean squared error (0.64 → 0.50) while the close inliers drop (12.7 % → 4.1 % within 5 cm) and the median distance grows. This is the classic failure mode that bounded correspondences and robust kernels address [24, 29].
- A ground vehicle on level ground should not move 0.22 m vertically or tilt by 7.5°. These components most likely absorb the different hand-set pitch corrections ($-10°$ vs. $-3°$).
- The fitness of 0.49974 m^2 (RMS 0.71 m) is large because of the unbounded matching, so it cannot serve as a quality measure here.

Robustness check. Re-registering the same preprocessed clouds with other standard settings (an experiment, not part of the project) gives:

Method	Time	Rotation	$-R^\top \boldsymbol{t}$ (fwd, up, lat) [m]	< 5 cm	< 10 cm
Project ICP (unbounded, 70 it.)	2.2 s	7.6°	$(0.543, -0.222, -0.233)$	4.1 %	10.8 %
ICP coarse-to-fine, $d_{\max} = 1 \to 0.1$ m	6.4 s	34.0° (yaw 33.7°)	$(1.077, 0.124, -2.342)$	22.2 %	34.5 %
GICP, $d_{\max} = 0.5$ m	0.9 s	7.8°	$(-0.190, 0.098, -0.014)$	13.5 %	20.2 %

Table 7: Three registrations of the same pair disagree strongly (measured).

The estimators disagree by metres and tens of degrees. This frame pair is ill-conditioned for registration: the scene is dominated by walls and ground, whose geometry constrains some motion directions poorly (the "corridor" degeneracy, analysed by Zhang et al. [41]), and the overlap is partial. The honest conclusion: the pipeline is implemented correctly, and the 0.59 m estimate is reproducible, but its accuracy cannot be established from this pair. To make it trustworthy one would: remove the ground by a RANSAC plane fit (levelling both scans automatically instead of $-10°/-3°$); bound correspondences and use a robust kernel; use point-to-plane ICP or GICP; restrict the motion to $SE(2)$; seed ICP with a wheel-odometry or IMU prior [3, 36]; and check the condition number of the ICP Hessian.

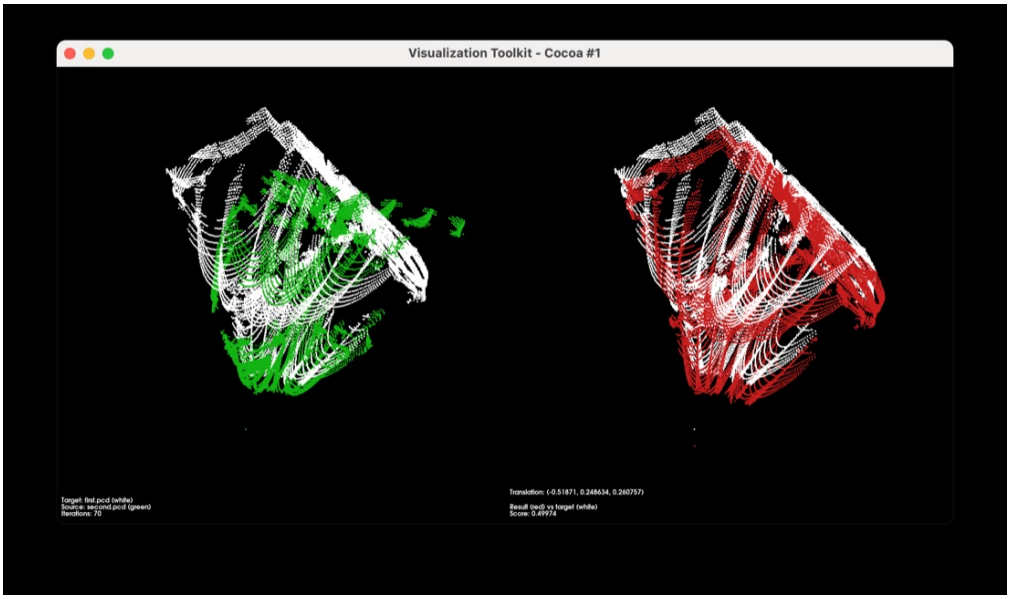


Figure 5: Odometry viewer from the original deliverable. Left: first (white) and second (green) after levelling; right: the ICP output.

10 Bonus — Interactive GUI

The viewer (`pclviewer.cpp`, `pclviewer.ui`) embeds a VTK render window in a Qt main window. It follows PCL's Qt tutorial and has five tabs: Setting (file paths, verbose, cloud toggles), Pos (lateral shift), SOR, ConeRec (cut bottom/top, fitness) and Line (left/right distance). Load reads and levels the cloud; Compute, or releasing any slider, reruns levels 1–4 and replaces the four displayed clouds (original green, cones red, obstacles cyan, racing line yellow).

Slider	Parameter	Scale	Range	Default
SOR	α (std. multiplier)	/100	0.5 – 2.0	1.1
Cut bottom	pass-through lower y	/10	-15 – -5 m	-10 m
Cut top	pass-through upper y	/10	-10 – 1.2 m	-0.2 m
Lateral	shift along z	/10	-1 – 1 m	0 m
Distance	left/right $d_{\max}$	/10	0.1 – 8 m	6 m
Fitness	ICP fitness threshold	/10000	0.0001 – 0.9	0.005

Table 8: GUI parameters. `QSlider` holds integers, so each value is a scaled integer.

Design choices to defend. Recomputation is triggered on `sliderReleased`, not `valueChanged`: a full recompute takes about 0.5 s (Section 12), so recomputing on every slider tick would freeze the UI. "Real-time" in the task means that results refresh as soon as a parameter is set. The pipeline runs on the GUI thread; moving it to a worker (`QtConcurrent::run` plus a `QFutureWatcher`) would keep the window responsive. The project already links Qt `Concurrent`.

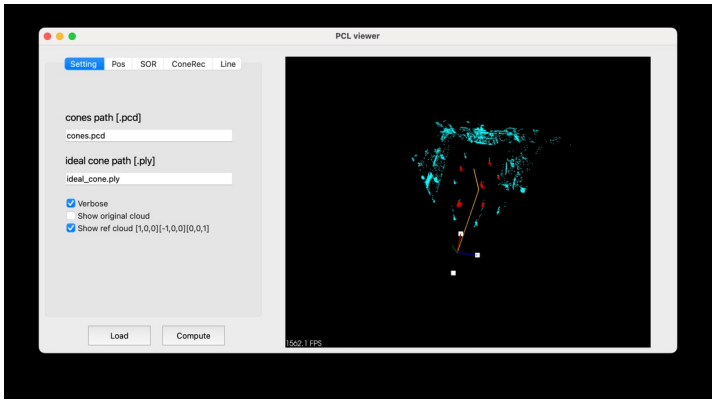


Figure 6: GUI after Compute: cones (red), obstacles (cyan), racing line (yellow) and the white reference points at $(\pm 1, 0, 0)$ and $(0, 0, 1)$.

11 C++: difficult patterns, explained

This section collects the parts of the code where the language does something non-obvious. References: Meyers [20], Stroustrup [34]; Eigen: Guennebaud et al. [13].

11.1 Shared pointers, aliasing and "const" that is not const

```
// PCL: PointCloud<T>::Ptr is std::shared_ptr<PointCloud<T>>
std::vector<int> detectCones(const pcl::PointCloud<pcl::PointXYZ>::Ptr &cloud, ...,
                             const pcl::PointCloud<pcl::PointXYZ>::Ptr &cone_model, ...)
{
    ...
    pcl::transformPointCloud(*cone_model, *cone_model, T_center_model); // modifies the model!
```

`const Ptr&` means the pointer cannot be reseated. The pointee is still mutable: `const std::shared_ptr<T>` is not `std::shared_ptr<const T>`. `detectCones` therefore re-centres the caller's model in place. This is idempotent here (the second centring is a no-op), but it is a hidden side effect. PCL offers `PointCloud<T>::ConstPtr` for read-only inputs. Passing `shared_ptr` by const reference avoids the atomic reference-count increment of a copy.

`get_cones_cloud(src, cones_cloud, ...)` fills its output parameter and returns the same pointer inside the result struct, so in the GUI `new_cone_cloud` and `cone_cloud` are two names for one object.

11.2 In-place filtering is safe in PCL

```
pass.setInputCloud(work_cloud);
pass.filter(*work_cloud);            // input and output are the same object
```

This looks like it reads and writes the same buffer, but `pcl::Filter::filter` detects it (`if (input_.get() == &output)`), filters into a temporary and then swaps. Without that check, the code would be undefined behaviour.

11.3 Keep-organised extraction: NaN as a tombstone

```
for (const auto &cluster : clusters) {
    pcl::PointIndices::Ptr cluster_ptr(new pcl::PointIndices(cluster));
    reg_grow_extract.setIndices(cluster_ptr);
    reg_grow_extract.setNegative(true);     // keep everything EXCEPT these indices
    reg_grow_extract.setKeepOrganized(true);// ...by writing NaN, not by erasing
    reg_grow_extract.filter(*dst_cloud);
}
```

All region indices were computed on the original cloud. Erasing the first region would shift every later index, so the second removal would delete the wrong points. Overwriting with NaN (a "tombstone") keeps every index valid. The price: later code must tolerate NaNs (`removeNaNFromPointCloud` produces an index list; the kd-tree skips non-finite points).

11.4 Structured bindings and aggregate returns (C++17)

```
struct ConeDetectionResults {
    pcl::PointCloud<pcl::PointXYZ>::Ptr centers_of_mass;
    pcl::PointCloud<pcl::PointXYZ>::Ptr cone_cloud;
};
auto [centers_of_mass, new_cone_cloud] = get_cones_cloud(working_cloud, cone_cloud, ...);
auto [src_left, src_right] = cones::distinguishLeftRight(centers_of_mass, 0, 0, -45, +45, 6);
```

The compiler creates one hidden object of the returned type, and the two names refer to its members in declaration order. Returning a small struct of `shared_ptr`s is cheap: copy elision is guaranteed for prvalues since C++17.

11.5 Argsort with `iota` + `sort` + a capturing lambda

```cpp
std::vector<int> idx(N);
std::iota(idx.begin(), idx.end(), 0);                // 0,1,...,N-1
std::sort(idx.begin(), idx.end(), [&](int a, int b) { // sort indices by distance
    const auto &pa = cones->points[a]; const auto &pb = cones->points[b];
    double da = (pa.x-robot_x)*(pa.x-robot_x) + (pa.y-robot_y)*(pa.y-robot_y);
    double db = (pb.x-robot_x)*(pb.x-robot_x) + (pb.y-robot_y)*(pb.y-robot_y);
    return da < db;
});
```

This sorts indices and leaves the data in place (NumPy's `argsort`). Comparing squared distances avoids $N \log N$ square roots and preserves the order. The comparator must be a strict weak ordering (<, never <=). `std::partial_sort` would suffice, since only the two smallest are needed.

11.6 `std::vector<char>` instead of `std::vector<bool>`

```cpp
std::vector<char> remaining(N, 1);
remaining[first_idx] = remaining[second_idx] = 0;
```

`std::vector<bool>` is a bit-packed specialisation whose `operator[]` returns a proxy object, not a `bool&`. `auto& r = v[i]` does not compile, `data()` does not exist, and concurrent writes to different elements race. `std::vector<char>` avoids all three.

11.7 Lambdas returning references need an explicit return type

```cpp
auto get_point = [&](int i) -> const pcl::PointXYZ& { return cones->points[i]; };
```

Without `-> const pcl::PointXYZ&`, the lambda's return type is deduced by `auto` rules, which drop references, so every call would copy the point. The reference is safe because `cones` outlives the lambda. `nearest_to`, `safe_turn` and `bad` are further local lambdas capturing by reference. Note that `rem_idx` is rebuilt each round and captured by reference, so each call sees the current set.

11.8 A C callback with `void*` user data (NLopt C API)

```cpp
static double objective_func(unsigned n, const double* x, double* grad, void* data) {
    auto* td = static_cast<TrackData*>(data);      // recover the type erased by void*
    ...
    if (grad) for (unsigned i = 0; i < n; ++i) grad[i] = 0.0; // LN_ algorithms pass nullptr
    return cost;
}
nlopt_opt opt = nlopt_create(NLOPT_LN_BOBYQA, N);
nlopt_set_min_objective(opt, objective_func, &td); // &td travels as void*
...
nlopt_destroy(opt);                                // must be called on EVERY return path
```

A C library cannot call a capturing lambda or a member function, so the problem data travel as `void*`, and `static_cast` restores the type. `static` gives the function internal linkage (not visible to other translation units). The manual `nlopt_destroy` is the C way of freeing resources. The RAII version cannot leak on an early return:

```cpp
std::unique_ptr<nlopt_opt_s, decltype(&nlopt_destroy)> opt(
    nlopt_create(NLOPT_LN_BOBYQA, N), &nlopt_destroy);
// or the C++ wrapper: nlopt::opt opt(nlopt::LN_BOBYQA, N);
```

Also worth noting: `objective_func` allocates a `vector<vector<double>>` on every evaluation. A flat `std::vector<double>` of size $2N$, or `Eigen::Matrix2Xd`, would avoid N heap allocations per call.

11.9 A function template with a dependent type

```cpp
template <typename T>
std::vector<T> linspace(T a, T b, size_t N) {
    T h = (b - a) / static_cast<T>(N-1);
    std::vector<T> xs(N);
    typename std::vector<T>::iterator x;   // 'typename': dependent name is a type
    T val;
    for (x = xs.begin(), val = a; x != xs.end(); ++x, val += h)  // comma operator
        *x = val;
    return xs;
}
```

`std::vector<T>::iterator` depends on `T`, so the compiler must be told that it is a type (`typename`). `T` is deduced as `float` from the call site. Accumulating `val += h` adds rounding error; `a + i*h` does not. $N = 1$ divides by zero.

11.10 Eigen: `rotate` post-multiplies

```
Eigen::Affine3f T = Eigen::Affine3f::Identity();
T.translation() << 0.0, 0.0, 0.1;                          // T = Trans
T.rotate(Eigen::AngleAxisf(M_PI/2, Eigen::Vector3f::UnitX())); // T = Trans * Rx
T.rotate(Eigen::AngleAxisf(-M_PI, Eigen::Vector3f::UnitY()));  // T = Trans * Rx * Ry
Eigen::Matrix3f R = transform.block<3,3>(0,0);              // fixed-size block view
Eigen::Matrix4d M = icp.getFinalTransformation().cast<double>();
```

`rotate(R)` computes $T \leftarrow TR$ (rotation applied first to the point); `prerotate(R)` computes $T \leftarrow RT$. Because the translation was set first and the rotations post-multiply, the translation is expressed in the output frame. That is why the 0.1 m ends up on z', the lateral axis. `block<3,3>(0,0)` is a compile-time-sized view with no copy until it is assigned. `compute3DCentroid` returns an `Eigen::Vector4f`, a homogeneous vector with $w = 1$.

11.11 Default arguments and duplicate definitions across files

`detectCones` has default arguments in both `cones_recog.h` and `cones_recog.cpp`. This compiles only because the `.cpp` does not include its own header. If it did, the compiler would report "redefinition of default argument". Defaults belong in the declaration (header) only. The values actually used are those passed by `get_cones_cloud` (-0.7, 0.2, 0.5, 13°), not the header defaults (-0.01, 0.3, 0.5, 20°). Similarly, `struct cones::Result` is defined both in the header and in the `.cpp`. The One Definition Rule allows identical class definitions in different translation units, but if the two ever diverge, the behaviour is undefined, with no diagnostic required.

11.12 Templates and precompiled instantiations: why Debug ≈ Release

`pcl::IterativeClosestPoint<pcl::PointXYZ, pcl::PointXYZ>` is a class template. PCL ships precompiled instantiations for common point types in `libpcl_registration`, built in Release by Homebrew. The project's own Debug flags therefore barely affect the heavy work. Measured: Debug $2{,}278$ ms vs. Release $2{,}260$ ms per ICP (0.8 %; Section 12).

11.13 Qt: signals, slots, moc and the generated UI

```
class PCLViewer : public QMainWindow {
    Q_OBJECT                        // moc generates the meta-object code (CMAKE_AUTOMOC)
public Q_SLOTS:
    void slidersReleased();
private:
    Ui::PCLViewer *ui;              // class generated by uic from pclviewer.ui (CMAKE_AUTOUIC)
};
connect(ui->horizontalSlider_sor, SIGNAL(sliderReleased()), this, SLOT(slidersReleased()));
// compile-time checked alternative:
connect(ui->horizontalSlider_sor, &QSlider::sliderReleased, this, &PCLViewer::slidersReleased);
```

The string-based `SIGNAL`/`SLOT` macros are resolved at run time: a typo prints a warning and the connection silently fails. The pointer-to-member form is checked by the compiler. `Ui::PCLViewer` is forward-declared and held by pointer, which keeps the generated header out of `pclviewer.h` (a compile firewall). It is deleted in the destructor; a `std::unique_ptr` would do this automatically.

11.14 Small traps that are easy to miss

- `static_cast<int>(initial_sor_std_dev_thr) * 100 + 100`: the cast binds to the variable first, so $1.1 \to 1$ and the result is 200, not 210.
- `pcl::console::TicToc time` is passed by value into `processClusterForConeDetection`, so each call copies the timer.
- `#if VTK_MAJOR_VERSION > 8` chooses the render-window API at compile time: VTK 9 needs a `vtkGenericOpenGLRenderWindow` attached with `setRenderWindow`, VTK 8 uses the older calls.
- In CMake, `find_package(Qt6 QUIET ...)` falls back to Qt5 and stores the choice in `QTX`, which is used later as `${QTX}::Widgets`. NLopt is found via `pkg_check_modules`.
- `std::getenv("PERCEPTION_HEADLESS")` enables the headless benchmark. It is checked after the `PCLVisualizer` is constructed, so headless runs still pay for creating the viewer.
- A log message for over-tall clusters prints "< min_cone_height" instead of "> max_cone_height" (copy-paste).

12 Statistics: what each number means and how it was obtained

12.1 Engineering footprint (from `STATS.md`)

Statistic	Value	How it is computed	Justification / caveat
Commits	89 (now 90)	`git rev-list --count HEAD`	89 at the snapshot; the "stats" commit itself is the 90th. 85 commits fall in the two-week window (28 Oct – 12 Nov 2025), 4 on 17 Nov 2025.
C++ files / headers	47 / 25	`git ls-files` filtered by extension	Whole repository, including R&D prototypes and branch copies in `other/`, not only delivered code.
GUI C++/header LOC	1,862	`wc -l` over the 14 files	Physical lines. 1,490 are non-blank, and about 115 are a commented-out test `main` in `racingline_nlopt.cpp`.
Odometry LOC	225	`wc -l main.cpp`	Before the stats commit; that commit added the 6-line headless switch, giving 231 today.
Delivered modules	7	count	cone recognition, cone cloud, obstacles, left-/right, racing line, viewer, odometry.
Qt UI files	2	`*.ui`	`pclviewer.ui` (used) and `oldpclviewer.ui` (legacy).
PCD / PLY assets	14 / 7	`git ls-files`	Includes duplicates across branches; there are 3 distinct task scans.

Table 9: Repository statistics, re-checked on 2026-10-07.

12.2 Runtime statistics (odometry)

Method. The odometry binary was run 10 times with `PERCEPTION_HEADLESS=1` under `/usr/bin/time -l`. ICP time comes from PCL's `TicToc`, which wraps only `icp.align()`. Wall, user and system time and peak resident set size (RSS) come from `time`. The run was repeated on 2026-10-07 in both build types to check reproducibility.

	STATS.md (Debug)	Reproduced (Debug)	Reproduced (Release)
ICP time, mean [ms]	2,291.18	2,278.47	2,259.64
ICP time, median [ms]	2,268.88	2,256.33	2,257.45
ICP time, min – max [ms]	2,253.55 – 2,401.99	2,208.04 – 2,424.61	2,185.85 – 2,346.50
standard deviation [ms]	—	73.7	45.5
Frequency $1000/\bar{t}$ [Hz]	0.436	0.439	0.443
CPU $=$ (user $+$ sys)/wall	94.9 %	94.6 %	95.4 %
user + sys / wall [s]	$30.49 + 0.72$ / 32.88	$30.64 + 0.61$ / 33.04	$30.26 + 0.59$ / 32.35
Peak RSS	115,982,336 B $=$ 110.6 MiB	112.7 MiB	111.1 MiB
Wall time per run [s]	3.29	3.30	3.24

Table 10: Odometry benchmark: original figures and reproduction (10 runs each, same machine).

How to read each number.

- **2,291 ms is ICP time per scan pair**, not per cloud and not the whole program. The other ≈1.0 s of each 3.3 s run is process start-up (loading the PCL/VTK shared libraries), reading two PCD files, two SOR passes and creating the viewer.
- **0.436 Hz** $= 1000/2291.18$. It is a throughput for this pair, not a sensor rate. A 10 Hz LiDAR leaves 100 ms per frame, so this ICP is about 23× too slow for real time.
- **94.9 % CPU** is (user $+$ sys)/wall in the `time` convention, where 100 % means one fully busy core. PCL was built without OpenMP (it warns at start-up), so the work is single-threaded: about 1/8 of the machine. The remaining 5 % is time blocked on I/O and library loading.
- **110.6 MiB** $= 115{,}982{,}336/2^{20}$ is the peak RSS. Two clouds of 24,000 `PointXYZ` take only $2 \times 24{,}000 \times 16$ B ≈ 0.73 MiB (PCL pads `PointXYZ` to 16 B for SIMD). Almost all of the footprint is shared libraries (PCL, VTK, FLANN) and the viewer.
- **Debug ≈ Release** (0.8 % apart) because the hot loop is in PCL's precompiled, optimised library code (Section 11).

- The run-to-run spread (SD 74 ms, about 3 %) is OS scheduling and thermal noise. The difference between 2,291 and 2,278 ms is within it.

12.3 Runtime statistics (cone pipeline)

Stage	Time [ms], mean $\pm$ SD	Complexity
Pass-through (y band)	0.1	$O(n)$
SOR ($k = 50$)	22.1 ± 1.1	$O(nk \log n)$
Normal estimation ($k = 50$)	20.1 ± 0.5	$O(nk \log n)$ + 3×3 eigen-solve per point
Region growing ($K = 300$)	180.3 ± 20.5	$O(nK \log n)$
Euclidean clustering ($r = 5$ cm)	6.4 ± 2.0	$O(n \log n)$
Gates + 9 ICP fits to the model	≈ 2	$O(I\, m \log M)$ per cluster, $m \leq 500$, $M = 150$
`get_cones_cloud` total	226.0 ± 7.2	
Left/right split	< 0.2	$O(N^2)$
Racing line (BOBYQA, 78 evaluations)	≈ 0.1	one $O(N)$ evaluation per iteration
Obstacle extraction	266.6 ± 20.8	dominated by its own region growing
Total	$\mathbf{492.9 \pm 28.0}$	$\Rightarrow \approx 2.0$ Hz

Table 11: Cone pipeline timing, 10 headless runs with GUI defaults, Release build (measured). Stage rows are from an instrumented replica of the same calls.

Region growing appears twice (walls for cones, walls for obstacles) and accounts for about 75 % of the total. The obvious optimisations: compute normals and regions once and share them; reduce $K = 300$; downsample with a voxel grid before normal estimation; or replace wall removal by ground-plane RANSAC plus a height band.

13 Limitations and what to do next

1. **Left/right uses one coordinate** (Section 7). Fix the frame mapping, then replace the greedy split by Delaunay-based path search or intensity-based cone colour.
2. **The curvature proxy is blind to reversals** (Section 8). Use the turning angle or Menger curvature; order the gates; draw all N optimised points.
3. **Odometry is ill-conditioned and unbounded** (Section 9). Bound correspondences, level the scans automatically with RANSAC, use point-to-plane ICP or GICP with a motion prior, and report the convergence state and inlier ratio instead of only “converged”.
4. **Thresholds tuned on one scene.** The yaw margin is 0.5°, and the fitness gate is inactive on this scene. Validating on more scans, with labelled cones and precision/recall, is the obvious next step.
5. **Not real-time.** 0.49 s for cones and 2.3 s for ICP, against a 0.1 s budget. Share normals, downsample, parallelise (OpenMP or TBB), and cache kd-trees.
6. **Code structure.** The report itself names organisation, readability and memory management as the weakest areas: free functions instead of classes, exported magic numbers, manual resource handling.
7. **Far-range distortion.** The report observes that cone spacing seems distorted at range. With $n \propto r^{-2}$, far cones have few points, so their centroids are noisier and biased towards the sensor. A covariance per cone would make this uncertainty explicit.

14 Question-and-answer drill

Why LiDAR and not the camera path?

LiDAR gives metric 3D directly: no monocular scale ambiguity, no lighting dependence, and cone geometry can be verified with a 3D model. The camera path would use ORB features [28], a brute-force matcher, the essential matrix $E = K^\top F K$ from the given intrinsics K (five-point RANSAC [11, 22]), and pose recovery up to an unknown scale [14]. LiDAR odometry gives translation in metres.

Why region growing instead of RANSAC to remove walls?

RANSAC removes one plane per run and needs to know how many planes to remove. Region growing with a 1° normal tolerance and a 500-point minimum removes all large planar regions at once. Cones cannot reach 500 points (the largest

has 113), so they are safe by construction.

Why Euclidean clustering and not K-means or DBSCAN?

K-means needs k in advance and assumes compact, similar-sized clusters. Euclidean clustering is DBSCAN with minPts $= 1$: no k, arbitrary shapes, and noise handled by the size limits [8, 500]. With a 5 cm radius at 3–5 m range it separates cones that are about 1 m apart.

What does a fitness of 0.005 mean physically?

It is a mean squared distance in m^2, so the RMS residual is $\sqrt{0.005} = 7.1$ cm. Accepted cones achieved 2.0–3.3 cm RMS. On this scene every candidate passed the fitness test; the yaw gate made the difference.

A cone is symmetric about its axis — why test yaw?

Yaw is a null direction of the cost for a true cone, so a local optimiser started at identity barely moves along it: accepted 1.2–12.5°. Wall fragments must be rotated to fit the model: rejected 17.5–45.3°. It is an empirical separator; 13° sits in the gap, and 15° would maximise the margin. A tilt check would be more principled.

How do you know the 6 detections are correct?

There are no labels. The evidence: the cluster heights (0.31–0.32 m for the best views) match the regulation cone; the residuals are 2–3 cm; the points follow $n \propto r^{-1.96}$ as predicted for fixed-size objects; and the six positions form a plausible bend that agrees with the MATLAB prototype's input. For a real evaluation I would label several scans and report precision and recall per range bin.

Explain the racing-line objective and the solver.

Offsets d_i along the normals of the centreline, bounded by the half-widths so that the line stays on track; cost $= \sum \kappa_i^2 + 0.1 \sum (\Delta d)^2$; curvature from finite differences (the h cancels; $\varepsilon = 10^{-6}$ guards against coincident points). BOBYQA is used because it is derivative-free and handles bounds natively, and N is tiny: 78 evaluations, 0.1 ms. On the toy track it beat CasADi/IPOPT (0.05223 vs. 0.05305).

Is the racing line on the real scene correct?

No, and I can explain why: the split orders the cones so that the centreline folds back, and the $\sin\theta$ proxy assigns zero cost to a 180° reversal, so the optimiser returns a folded line with $J \approx 0$. Fixes: correct the left/right frame bug, order the gates along the track, and use the signed turning angle or Menger curvature. The optimisation machinery itself is validated on the toy track.

Is the odometry result reliable?

It is reproducible (identical numbers across runs and machines), and the conversion to a displacement is correct: $-R^\top \boldsymbol{t}$, about 0.54 m forward and 0.23 m lateral. But ICP hit its 70-iteration cap with unbounded correspondences, close inliers fell from 12.7 % to 4.1 %, and GICP and bounded ICP give very different answers. So the pair is ill-conditioned. I would bound correspondences, level automatically, use GICP or point-to-plane ICP with a motion prior, and check the Hessian conditioning.

Is it real-time? Where would you optimise?

No: 0.49 s for cones (2 Hz) and 2.3 s for ICP (0.44 Hz), against 100 ms at 10 Hz. Region growing is about 75 % of the cone time and runs twice, so share it, downsample, and lower K. For ICP: voxel-downsample to a few thousand points, bound correspondences (fewer iterations), use OpenMP (PCL was built without it), or switch to a tuned point-to-point pipeline such as KISS-ICP [38].

Why is a Debug build as fast as Release?

The time is spent inside PCL's precompiled template instantiations (`IterativeClosestPoint<PointXYZ,PointXYZ>`, FLANN kd-tree), which are always optimised. Our own code is a thin layer. Measured difference: 0.8 %.

Point-to-point vs. point-to-plane ICP?

Point-to-point minimises Euclidean distances between matched points and has an exact SVD solution per iteration. Point-to-plane minimises the distance along the target normal, so points may slide along surfaces. It converges in fewer iterations on

man-made scenes but needs normals and a linearised solve [7, 29].

What would you change first with one more week?

(1) Fix the left/right frame and use Delaunay midpoints; (2) make the curvature reversal-aware; (3) bounded, robust, levelled registration with an $SE(2)$ constraint; (4) wrap the pipeline in classes with a parameter struct shared by the GUI and the CLI, plus unit tests per stage; (5) run the computation on a worker thread.

15 Formula sheet

Inverse transform	$T^{-1} = \begin{bmatrix} R^\top & -R^\top \boldsymbol{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}$
Rotation angle / axis	$\theta = \arccos \frac{\operatorname{tr} R - 1}{2}, \quad \boldsymbol{u} = \frac{1}{2\sin\theta}(r_{32} - r_{23},\, r_{13} - r_{31},\, r_{21} - r_{12})$
SOR	keep $\bar{d}_i \le \mu + \alpha\sigma, \quad \bar{d}_i = \frac{1}{k}\sum_{j \in \mathcal{N}_k(i)} \lVert \boldsymbol{p}_i - \boldsymbol{p}_j \rVert$
Normal, curvature	$\boldsymbol{n} = \boldsymbol{v}_0(C), \quad \sigma = \lambda_0/(\lambda_0 + \lambda_1 + \lambda_2) \in [0, \frac{1}{3}]$
Region growing	join if $\lvert \boldsymbol{n}_s \cdot \boldsymbol{n}_q \rvert \ge \cos\theta_{\text{th}}$; seed if $\sigma < c_{\text{th}}$
ICP step	$H = \sum (\boldsymbol{p}_i - \bar{\boldsymbol{p}})(\boldsymbol{q}_i - \bar{\boldsymbol{q}})^\top = U\Sigma V^\top, \; R = V \operatorname{diag}(1, 1, \det VU^\top) U^\top, \; \boldsymbol{t} = \bar{\boldsymbol{q}} - R\bar{\boldsymbol{p}}$
Fitness	$F = \frac{1}{n}\sum \min_j \lVert T\boldsymbol{p}_i - \boldsymbol{q}_j \rVert^2$ [m^2], RMS $= \sqrt{F}$
Yaw (about y)	$\psi = \operatorname{atan2}(r_{13}, r_{33})$
Turning angle	$\delta = \operatorname{wrap}\big(\operatorname{atan2}(\boldsymbol{b}) - \operatorname{atan2}(\boldsymbol{a})\big)$, or $\operatorname{atan2}(\boldsymbol{a} \times \boldsymbol{b},\; \boldsymbol{a} \cdot \boldsymbol{b})$
Discrete curvature	$\kappa = \dfrac{\boldsymbol{a} \times \boldsymbol{b}}{(\lVert \boldsymbol{a} \rVert^2 + \varepsilon)^{3/2}}$; Menger $= \dfrac{2\sin\theta}{\lVert \boldsymbol{p}_{i+1} - \boldsymbol{p}_{i-1} \rVert}$
Racing line	$\min_{\lvert d_i \rvert \le w_i} \sum \kappa_i^2 + \lambda \sum (d_{i+1} - d_i)^2, \; \boldsymbol{p}_i = \boldsymbol{m}_i + d_i \boldsymbol{n}_i$
Odometry	$\boldsymbol{p}^{(2)} = T\boldsymbol{p}^{(1)} \Rightarrow$ displacement $= -R^\top \boldsymbol{t}$
Density	$n \propto r^{-2}$ (fitted exponent -1.96)
Throughput, CPU	$f = 1000/\bar{t}_{\text{ms}}$ Hz; CPU $=$ (user $+$ sys)/wall (100 % = one core)

Table 12: Formulas to have at your fingertips.

References

[1] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl. CasADi: A software framework for nonlinear optimization and optimal control. Mathematical Programming Computation, 11(1):1–36, 2019.

[2] K. S. Arun, T. S. Huang, and S. D. Blostein. Least-squares fitting of two 3-D point sets. IEEE Transactions on Pattern Analysis and Machine Intelligence, 9(5):698–700, 1987.

[3] T. D. Barfoot. State Estimation for Robotics. Cambridge University Press, 2017.

[4] J. L. Bentley. Multidimensional binary search trees used for associative searching. Communications of the ACM, 18(9):509–517, 1975.

[5] P. J. Besl and N. D. McKay. A method for registration of 3-D shapes. IEEE Transactions on Pattern Analysis and Machine Intelligence, 14(2):239–256, 1992.

[6] P. Biber and W. Straßer. The normal distributions transform: A new approach to laser scan matching. In Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 2743–2748, 2003.

[7] Y. Chen and G. Medioni. Object modelling by registration of multiple range images. Image and Vision Computing, 10(3):145–155, 1992.

[8] A. R. Conn, K. Scheinberg, and L. N. Vicente. Introduction to Derivative-Free Optimization. SIAM, 2009.

[9] A. Dhall, D. Dai, and L. Van Gool. Real-time 3D traffic cone detection for autonomous driving. In Proc. IEEE Intelligent Vehicles Symposium (IV), pages 494–501, 2019.

[10] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In Proc. 2nd International Conference on Knowledge Discovery and Data Mining (KDD), pages 226–231, 1996.

[11] M. A. Fischler and R. C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. Communications of the ACM, 24(6):381–395, 1981.

[12] N. Gosala et al. Redundant perception and state estimation for reliable autonomous racing. In Proc. IEEE International Conference on Robotics and Automation (ICRA), pages 6561–6567, 2019.

[13] G. Guennebaud, B. Jacob, et al. Eigen v3. `https://eigen.tuxfamily.org`, 2010.

[14] R. Hartley and A. Zisserman. Multiple View Geometry in Computer Vision. Cambridge University Press, 2nd edition, 2004.

[15] A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann. Minimum curvature trajectory planning and control for an autonomous race car. Vehicle System Dynamics, 58(10):1497–1527, 2020.

[16] H. Hoppe, T. DeRose, T. Duchamp, J. McDonald, and W. Stuetzle. Surface reconstruction from unorganized points. In Proc. SIGGRAPH, pages 71–78, 1992.

[17] B. K. P. Horn. Closed-form solution of absolute orientation using unit quaternions. Journal of the Optical Society of America A, 4(4):629–642, 1987.

[18] J. Kabzan et al. AMZ driverless: The full autonomous racing system. Journal of Field Robotics, 37(7):1267–1294, 2020.

[19] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom. PointPillars: Fast encoders for object detection from point clouds. In Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 12697–12705, 2019.

[20] S. Meyers. Effective Modern C++. O'Reilly Media, 2014.

[21] M. Muja and D. G. Lowe. Scalable nearest neighbor algorithms for high dimensional data. IEEE Transactions on Pattern Analysis and Machine Intelligence, 36(11):2227–2240, 2014.

[22] D. Nistér. An efficient solution to the five-point relative pose problem. IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(6):756–770, 2004.

[23] M. Pauly, M. Gross, and L. P. Kobbelt. Efficient simplification of point-sampled surfaces. In Proc. IEEE Visualization, pages 163–170, 2002.

[24] F. Pomerleau, F. Colas, and R. Siegwart. A review of point cloud registration algorithms for mobile robotics. Foundations and Trends in Robotics, 4(1):1–104, 2015.

[25] M. J. D. Powell. The BOBYQA algorithm for bound constrained optimization without derivatives. Technical Report NA2009/06, Department of Applied Mathematics and Theoretical Physics, University of Cambridge, 2009.

[26] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 652–660, 2017.

[27] T. Rabbani, F. A. van den Heuvel, and G. Vosselman. Segmentation of point clouds using smoothness constraint. In ISPRS Commission V Symposium, International Archives of Photogrammetry, Remote Sensing and Spatial Information Sciences, volume 36, pages 248–253, 2006.

[28] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski. ORB: An efficient alternative to SIFT or SURF. In Proc. IEEE International Conference on Computer Vision (ICCV), pages 2564–2571, 2011.

[29] S. Rusinkiewicz and M. Levoy. Efficient variants of the ICP algorithm. In Proc. 3rd International Conference on 3-D Digital Imaging and Modeling (3DIM), pages 145–152, 2001.

[30] R. B. Rusu. Semantic 3D Object Maps for Everyday Manipulation in Human Living Environments. PhD thesis, Technische Universität München, 2009.

[31] R. B. Rusu and S. Cousins. 3D is here: Point Cloud Library (PCL). In Proc. IEEE International Conference on Robotics and Automation (ICRA), 2011.

[32] R. B. Rusu, Z. C. Marton, N. Blodow, M. Dolha, and M. Beetz. Towards 3D point cloud based object maps for household environments. Robotics and Autonomous Systems, 56(11):927–941, 2008.

[33] A. Segal, D. Hähnel, and S. Thrun. Generalized-ICP. In Robotics: Science and Systems (RSS), 2009.

[34] B. Stroustrup. The C++ Programming Language. Addison-Wesley, 4th edition, 2013.

[35] R. Szeliski. Computer Vision: Algorithms and Applications. Springer, 2nd edition, 2022.

[36] S. Thrun, W. Burgard, and D. Fox. Probabilistic Robotics. MIT Press, 2005.

[37] S. Umeyama. Least-squares estimation of transformation parameters between two point patterns. IEEE Transactions on Pattern Analysis and Machine Intelligence, 13(4):376–380, 1991.

[38] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss. KISS-ICP: In defense of point-to-point ICP – simple, accurate, and robust registration if done the right way. IEEE Robotics and Automation Letters, 8(2):1029–1036, 2023.

[39] A. Wächter and L. T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Mathematical Programming, 106(1):25–57, 2006.

[40] J. Zhang and S. Singh. LOAM: Lidar odometry and mapping in real-time. In Robotics: Science and Systems (RSS), 2014.

[41] J. Zhang, M. Kaess, and S. Singh. On degeneracy of optimization-based state estimation problems. In Proc. IEEE International Conference on Robotics and Automation (ICRA), pages 809–816, 2016.

[42] Y. Zhou and O. Tuzel. VoxelNet: End-to-end learning for point cloud based 3D object detection. In Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 4490–4499, 2018.